

Laporan Praktikum Kontrol Cerdas

Scitik Learn

Nama : Aryo Damarjati
Nim : 234308063
Kelas : TKA 6C
Akun Github : <https://github.com/aryodamar234308063tka-cmd>

Abstrak

Praktikum ini bertujuan untuk mengimplementasikan konsep kontrol cerdas berbasis machine learning menggunakan library Scikit-learn dalam lingkungan pemrograman Python. Tahapan praktikum meliputi proses pengumpulan dataset citra menggunakan OpenCV, ekstraksi fitur landmark tangan dan tubuh menggunakan MediaPipe, pelatihan model klasifikasi dengan algoritma Random Forest, serta implementasi sistem secara real-time melalui webcam. Dataset yang diperoleh diproses melalui tahap normalisasi dan pembagian data latih serta data uji untuk mengevaluasi performa model. Selain klasifikasi gestur tangan, sistem juga mengimplementasikan deteksi postur duduk dan berdiri menggunakan perhitungan sudut berbasis koordinat landmark tubuh. Hasil praktikum menunjukkan bahwa pendekatan machine learning mampu mengklasifikasikan gestur dengan tingkat akurasi yang baik, sementara metode rule-based berbasis sudut efektif untuk klasifikasi postur sederhana. Secara keseluruhan, sistem yang dikembangkan membuktikan bahwa integrasi computer vision dan machine learning dapat mendukung pengembangan sistem kontrol cerdas yang adaptif, real-time, dan aplikatif pada berbagai bidang otomasi dan interaksi manusia-komputer.

Kata kunci: Kontrol Cerdas, Machine Learning, Scikit-learn, MediaPipe, OpenCV, Klasifikasi Gestur, Deteksi Postur.

1. Pendahuluan

Perkembangan teknologi *kecerdasan buatan* (*Artificial Intelligence/AI*) dan *machine learning* (ML) telah mendorong penerapan model cerdas dalam berbagai sistem kontrol untuk meningkatkan performa, adaptabilitas, dan kecepatan

pengambilan keputusan tanpa ketergantungan pada aturan statis tradisional. Sistem kontrol cerdas merupakan gabungan antara teori kontrol klasik dan teknik pembelajaran data-driven yang memungkinkan sistem adaptif terhadap kondisi lingkungan yang dinamis dan tidak pasti. Konsep ini memainkan peran penting dalam otomasi industri, robotika, *smart grid*, dan sistem otonom lainnya karena kemampuannya dalam menangani non-linearitas dan ketidakpastian yang sulit diatasi oleh kontrol linear konvensional (Zhengyu, 2025).

Dalam konteks pembelajaran mesin, alat seperti *Scikit-Learn* library Python untuk algoritma *machine learning* menyediakan sekumpulan teknik pembelajaran terawasi (*supervised learning*) dan tidak terawasi (*unsupervised learning*) yang dapat diaplikasikan dalam perancangan kontrol cerdas, seperti klasifikasi untuk mendeteksi kondisi sistem, regresi untuk prediksi parameter kontrol, serta evaluasi dan validasi model untuk optimasi performa. Dengan pendekatan pembelajaran berbasis data, *learning algorithms* mampu menyesuaikan parameter kontrol secara real-time berdasarkan umpan balik data yang terus diperbarui, sehingga memperbaiki respons sistem terhadap gangguan dan variasi lingkungan yang kompleks (Abo Elkhair dkk., 2025)

Penelitian yang dilakukan oleh menunjukkan tren kuat dalam mengintegrasikan teknik *machine learning (ML)* ke dalam sistem kontrol cerdas (*intelligent control*), khususnya melalui pengembangan model hybrid yang menggabungkan pembelajaran data dengan logika kontrol konvensional untuk meningkatkan akurasi, adaptivitas, dan ketahanan terhadap variasi operasional. Pendekatan seperti ini tidak hanya diterapkan pada *microgrid* dan sistem energi terbarukan untuk optimasi dan stabilitas jaringan, tetapi juga pada otomasi proses industri dan sistem robotik, sehingga memperluas cakupan aplikasi kontrol cerdas berbasis AI untuk menghadapi tantangan dinamis serta meningkatkan otomatisasi dan pengambilan keputusan secara real-time dalam berbagai domain teknologi maju (Akbulut dkk., 2024).

Dengan demikian, praktikum ini bertujuan mengeksplorasi implementasi teknik kontrol cerdas menggunakan *Scikit-Learn* sebagai platform *machine*

learning untuk mempelajari bagaimana model-model belajar dapat meningkatkan kemampuan sistem kontrol dalam konteks yang lebih nyata dan aplikatif.

2. Tujuan :

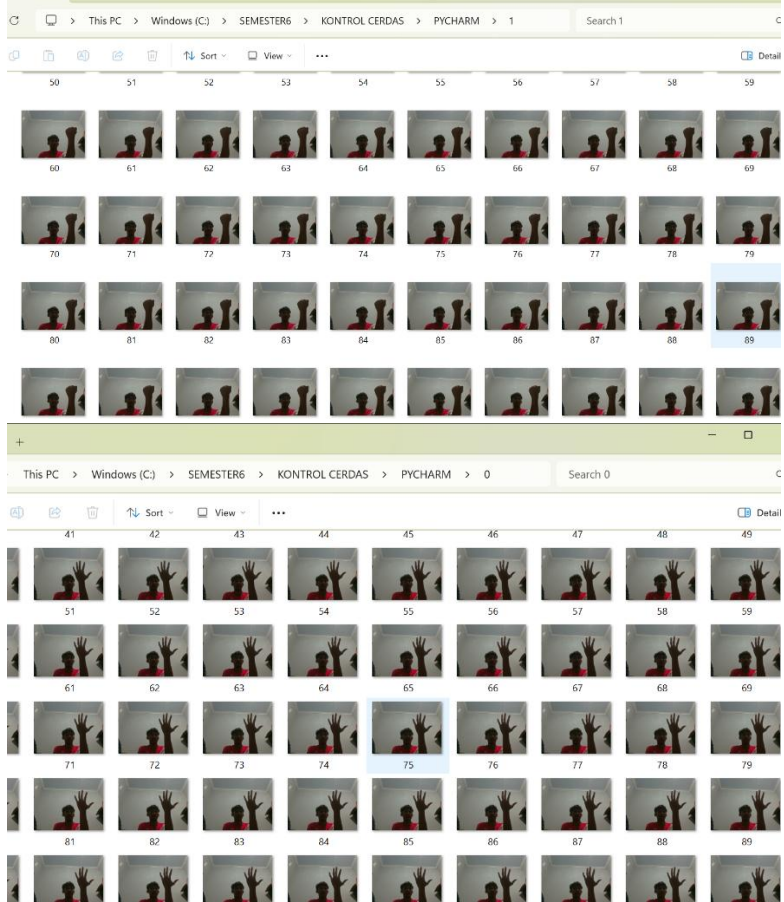
- Memahami konsep dasar machine learning secara praktis.
- Mempelajari penggunaan library Scikit-learn dalam pemodelan data.
- Mengimplementasikan algoritma klasifikasi, regresi, dan clustering.
- Melatih kemampuan preprocessing dan pembagian data (train–test).
- Menganalisis performa model menggunakan metode evaluasi.
- Mengaplikasikan machine learning untuk menyelesaikan permasalahan nyata.

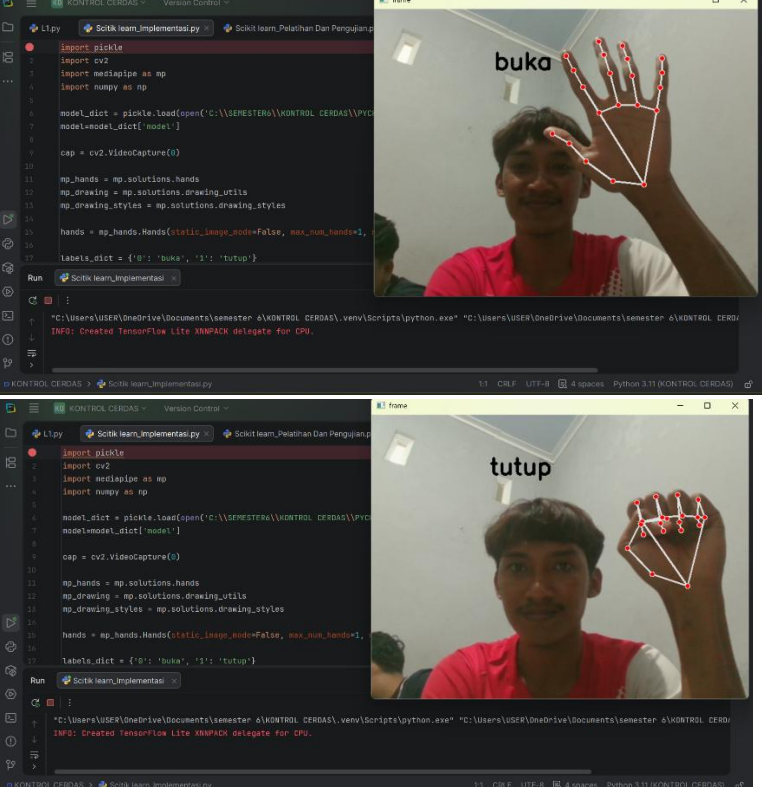
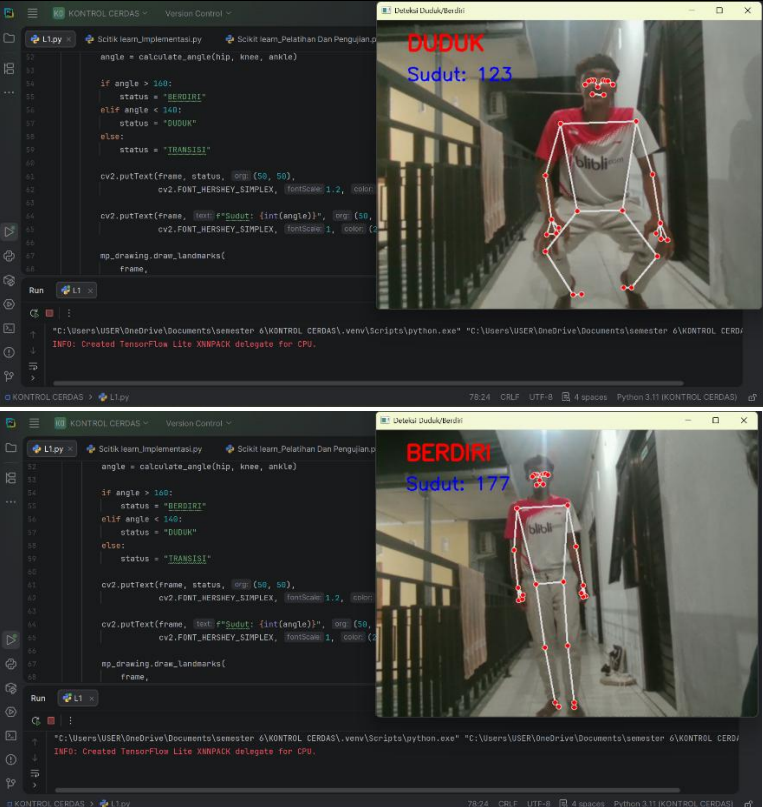
3. Manfaat :

- Memahami penerapan machine learning menggunakan Scikit-Learn.
- Melatih pengolahan dan klasifikasi data gesture.
- Meningkatkan kemampuan pemrograman Python.
- Mengembangkan keterampilan analisis hasil model.
- Menambah wawasan penerapan sistem deteksi gesture dalam bidang teknologi.

4. Program

No	Variabel	Hasil percobaan
.		

1	Pengumpulan Data menggunakan webcam					
2	Ekstraksi Data dari Dataset	<table><tr><td>data.pickle</td><td>25/02/2026 21:39</td><td>PICKLE File</td><td>70 KB</td></tr></table>	data.pickle	25/02/2026 21:39	PICKLE File	70 KB
data.pickle	25/02/2026 21:39	PICKLE File	70 KB			
3	Pelatihan dan Pengujian	<table><tr><td>model.p</td><td>25/02/2026 21:39</td><td>P File</td><td>58 KB</td></tr></table>	model.p	25/02/2026 21:39	P File	58 KB
model.p	25/02/2026 21:39	P File	58 KB			

4	Implementasi	 The screenshot shows a Python script in a Jupyter Notebook environment. The script imports necessary libraries (cv2, mediapipe, numpy) and sets up a video capture. It uses the MediaPipe Hands module to detect hands in real-time. The output displays two frames: one with the hand open, labeled 'buka', and another with the hand closed, labeled 'tutup'. The hand is overlaid with a skeletal model showing joints and connections.
5.	Latihan B	 The screenshot shows a Python script in a Jupyter Notebook environment. The script imports necessary libraries (cv2, mediapipe, numpy) and sets up a video capture. It uses the MediaPipe Pose module to detect a person's pose in real-time. The output displays two frames: one with a person in a crouching position, labeled 'DUDUK' with an angle of 123, and another with a person in a standing position, labeled 'BERDIRI' with an angle of 177. The person is overlaid with a skeletal model showing joints and connections.

5. Analisis Program

1. Pengumpulan Data menggunakan webcam

Percobaan pertama merupakan implementasi pengambilan dataset citra menggunakan library OpenCV pada Python untuk kebutuhan sistem kontrol cerdas berbasis pengolahan citra. Program diawali dengan mengimpor library os untuk manajemen direktori, sleep dari modul time, serta cv2 dari OpenCV untuk mengakses kamera dan memproses gambar. Variabel DATA_DIR digunakan untuk menentukan lokasi penyimpanan dataset, yaitu pada folder SEMESTER6/KONTROL CERDAS/PYCHARM. Parameter number_of_classes = 2 menunjukkan bahwa data akan dikumpulkan dalam dua kelas berbeda, sedangkan dataset_size = 100 menandakan setiap kelas akan memiliki 100 gambar. Kamera diaktifkan menggunakan cv2.VideoCapture(0) yang mengakses kamera utama perangkat. Pada proses utama, program melakukan perulangan sebanyak jumlah kelas. Untuk setiap kelas, sistem membuat subfolder berdasarkan indeks kelas (0 dan 1). Sebelum pengambilan gambar dimulai, layar akan menampilkan pesan “Ready? Press ‘Q’!” sebagai instruksi kepada pengguna. Setelah tombol “Q” ditekan, proses pengambilan gambar dimulai secara otomatis. Selanjutnya, program mengambil frame dari kamera secara berulang hingga mencapai 100 gambar untuk setiap kelas. Setiap frame disimpan dalam format JPG dengan nama file berdasarkan nomor urut (0.jpg hingga 99.jpg). Setelah seluruh data terkumpul, kamera dilepas dan seluruh jendela ditutup. Secara keseluruhan, kode ini berfungsi sebagai tahap akuisisi data untuk pelatihan model machine learning berbasis citra.

2. Ekstraksi Data dari Datashet

Percobaan ke 2 merupakan tahap praproses data (preprocessing) untuk membangun dataset berbasis fitur koordinat tangan menggunakan library MediaPipe dan OpenCV. Program diawali dengan mengimpor modul os untuk manajemen folder, pickle untuk penyimpanan data, mediapipe untuk ekstraksi landmark tangan, serta cv2 untuk membaca citra. Model mp.solutions.hands diinisialisasi dalam mode gambar statis (static_image_mode=True) dengan batas maksimal satu tangan (max_num_hands=1), sehingga setiap gambar diproses secara independen. Program kemudian membaca seluruh folder kelas pada

direktori dataset. Setiap folder merepresentasikan label tertentu (misalnya kelas 0 dan 1). Untuk setiap gambar dalam folder tersebut, sistem membaca file menggunakan `cv2.imread`. Jika gambar tidak dapat dibaca, proses dilewati. Gambar yang berhasil dibaca dikonversi dari format BGR ke RGB sebelum diproses oleh MediaPipe. Apabila tangan terdeteksi (`results.multi_hand_landmarks`), sistem akan mengekstrak koordinat x dan y dari setiap landmark. Koordinat tersebut kemudian dinormalisasi dengan cara mengurangkan nilai minimum x dan y pada setiap gambar, sehingga posisi tangan menjadi relatif terhadap titik terendahnya. Proses ini bertujuan mengurangi pengaruh posisi absolut tangan dalam frame dan membuat fitur lebih konsisten untuk pelatihan model machine learning. Seluruh data fitur disimpan dalam list data, sedangkan label kelas disimpan dalam list labels. Pada akhir proses, dataset disimpan dalam file `data.pickle` menggunakan pickle, sehingga dapat digunakan pada tahap pelatihan model klasifikasi.

3. Pelatihan dan Pengujian

Percobaan ke 3 merupakan tahap pelatihan dan evaluasi model klasifikasi pada sistem kontrol cerdas berbasis machine learning. Program diawali dengan memuat dataset yang telah disimpan sebelumnya dalam file `data.pickle` menggunakan modul pickle. Data yang dimuat terdiri dari dua bagian, yaitu fitur (`data`) yang berisi koordinat landmark tangan hasil ekstraksi sebelumnya, serta label (`labels`) yang merepresentasikan kelas masing-masing data. Kedua data tersebut kemudian dikonversi ke dalam bentuk array menggunakan pustaka NumPy agar dapat diproses oleh algoritma machine learning. Selanjutnya, dataset dibagi menjadi data latih dan data uji menggunakan fungsi `train_test_split` dari scikit-learn dengan proporsi 80% untuk pelatihan dan 20% untuk pengujian. Parameter `shuffle=True` memastikan data diacak terlebih dahulu agar distribusi kelas lebih merata dan model tidak bias terhadap urutan data. Model yang digunakan adalah `RandomForestClassifier`, yaitu algoritma berbasis ensemble learning yang membangun banyak pohon keputusan dan menggabungkan hasil prediksinya untuk meningkatkan akurasi serta mengurangi overfitting. Model kemudian dilatih menggunakan data latih melalui metode `fit()`. Setelah proses pelatihan selesai, model diuji menggunakan

data uji untuk menghasilkan prediksi. Tingkat akurasi dihitung menggunakan `accuracy_score`, yang menunjukkan persentase sampel yang berhasil diklasifikasikan dengan benar. Nilai akurasi dicetak dalam bentuk persentase sebagai indikator performa model. Terakhir, model yang telah dilatih disimpan ke dalam file `model.p` menggunakan `pickle`, sehingga dapat digunakan kembali tanpa perlu melakukan pelatihan ulang.

4. Implementasi

Percobaan ke 4 merupakan tahap implementasi sistem klasifikasi gestur tangan secara real-time menggunakan kamera. Model yang telah dilatih sebelumnya dimuat dari file `model.p` menggunakan modul `pickle`, kemudian digunakan untuk melakukan prediksi terhadap data baru yang ditangkap langsung dari kamera. Program memanfaatkan library `OpenCV` untuk mengakses webcam melalui `cv2.VideoCapture(0)` dan menampilkan hasil deteksi pada layar. Selain itu, digunakan `MediaPipe` khususnya modul `mp.solutions.hands` untuk mendeteksi serta melacak landmark tangan secara real-time. Parameter `min_detection_confidence` dan `min_tracking_confidence` diatur sebesar 0.7 untuk meningkatkan akurasi dan stabilitas deteksi. Dalam setiap frame yang tertangkap, gambar dikonversi dari format BGR ke RGB sebelum diproses oleh `MediaPipe`. Jika tangan terdeteksi, sistem menggambar titik dan koneksi landmark pada layar. Selanjutnya, koordinat setiap landmark diekstraksi dan dinormalisasi menggunakan metode bounding box (mengurangi nilai minimum dan membagi dengan rentang maksimum–minimum). Normalisasi ini bertujuan agar model lebih robust terhadap perbedaan posisi dan skala tangan di dalam frame. Data fitur yang telah diproses kemudian dimasukkan ke model untuk dilakukan prediksi. Hasil prediksi dikonversi menjadi label teks berdasarkan `labels_dict`, yaitu “buka” atau “tutup”. Teks hasil klasifikasi ditampilkan pada layar secara langsung. Program berjalan terus-menerus hingga pengguna menekan tombol “q”, kemudian kamera dilepas dan jendela ditutup. Secara keseluruhan, kode ini merepresentasikan tahap implementasi sistem kontrol cerdas berbasis pengenalan gestur tangan secara real-time.

5. Latihan B

Percobaan ke 5 program dimulai dengan inisialisasi modul pose dari MediaPipe untuk mendeteksi titik-titik landmark tubuh manusia. Kamera diakses menggunakan `cv2.VideoCapture(0)` yang berarti sistem mengambil input langsung dari webcam. Setiap frame yang tertangkap dikonversi dari format BGR ke RGB karena model pose pada MediaPipe bekerja pada format RGB. Fungsi `calculate_angle()` digunakan untuk menghitung besar sudut antara tiga titik koordinat (hip, knee, dan ankle) menggunakan pendekatan trigonometri berbasis arctangent. Perhitungan ini menghasilkan sudut lutut dalam satuan derajat yang kemudian dinormalisasi agar berada pada rentang 0–180 derajat. Dalam proses utama (looping), sistem membaca landmark tubuh bagian kanan, yaitu pinggul (RIGHT_HIP), lutut (RIGHT_KNEE), dan pergelangan kaki (RIGHT_ANKLE). Sudut lutut dihitung untuk menentukan kondisi postur. Jika sudut lebih dari 160° , sistem mengklasifikasikan sebagai “BERDIRI”. Jika kurang dari 140° , dikategorikan sebagai “DUDUK”. Sedangkan nilai di antara keduanya dianggap sebagai “TRANSISI”. Hasil klasifikasi dan nilai sudut ditampilkan langsung pada frame video menggunakan fungsi `cv2.putText()`. Selain itu, kerangka tubuh juga divisualisasikan menggunakan `mp_drawing.draw_landmarks()` sehingga pengguna dapat melihat hasil deteksi pose secara visual. Secara keseluruhan, kode ini menunjukkan penerapan konsep computer vision dan perhitungan sudut berbasis koordinat untuk klasifikasi postur sederhana berbasis rule-based (threshold sudut), tanpa menggunakan machine learning classifier tambahan. Sistem ini bekerja secara real-time dan bergantung pada akurasi deteksi landmark oleh MediaPipe Pose.

6. Kesimpulan

- Praktikum berhasil mengimplementasikan konsep kontrol cerdas berbasis machine learning menggunakan Scikit-learn.
- Proses akuisisi data menggunakan webcam berjalan efektif sebagai tahap pembentukan dataset.
- Ekstraksi fitur landmark dengan MediaPipe meningkatkan representasi data untuk proses klasifikasi.

- Algoritma Random Forest mampu memberikan performa klasifikasi yang baik pada data gestur tangan.
- Sistem implementasi real-time menunjukkan integrasi yang baik antara model terlatih dan deteksi kamera.
- Metode rule-based berbasis perhitungan sudut efektif untuk klasifikasi postur duduk dan berdiri.
- Sistem secara keseluruhan mampu berjalan secara real-time dan responsif terhadap input pengguna.
- Percobaan 1 sampai dengan 4 merupakan alur dari proses pengumpulan data, ekstraksi data, pelatihan dan pengujian, serta implementasi.

7. Saran

Pengembangan sistem dapat ditingkatkan dengan menambah jumlah dataset agar model memiliki generalisasi yang lebih baik terhadap variasi posisi dan pencahayaan. Selain itu, penggunaan algoritma lain seperti Support Vector Machine atau Neural Network dapat dibandingkan untuk memperoleh performa yang lebih optimal. Pada deteksi postur, sistem dapat dikembangkan menggunakan pendekatan machine learning agar klasifikasi lebih adaptif dibandingkan metode berbasis threshold sudut tetap. Optimalisasi parameter deteksi dan peningkatan kualitas pencahayaan kamera juga disarankan untuk meningkatkan stabilitas dan akurasi sistem secara keseluruhan.

8. Referensi

- Abo Elkhair, E., Abu Elanien, A. E. B., Mahmoud, G. M., & ElRefaie, H. B. (2025). Enhancing PI control in microgrids using machine-learning techniques. *Scientific Reports*, 15(1), 38129. <https://doi.org/10.1038/s41598-025-20781-5>
- Akbulut, O., Cavus, M., Cengiz, M., Allahham, A., Giaouris, D., & Forshaw, M. (2024). Hybrid Intelligent Control System for Adaptive Microgrid Optimization: Integration of Rule-Based Control and Deep Learning Techniques. *Energies*, 17(10), 2260. <https://doi.org/10.3390/en17102260>
- Zhengyu. (2025). *A Review on Intelligent Control*.

